

Week-1: Design and Patterns and Principles

1) Singleton Pattern:

- ❑ Singleton Pattern ensures that only one instance of a class is created throughout the application.
- ❑ The constructor is declared private to prevent object creation from outside the class.
- ❑ A static variable is used to store the single instance of the class.
- ❑ A public static method (GetInstance()) provides global access to the single instance.
- ❑ Singleton is commonly used for database connections, logging services, configuration settings, and cache management.

```
1  using System;
2  public sealed class Singleton{
3      private static Singleton instance;
4      private Singleton(){
5          Console.WriteLine("Singleton Instance Created");
6      }
7      public static Singleton GetInstance(){
8          if (instance == null){
9              instance = new Singleton();
10         }
11         return instance;
12     }
13     public void Display(){
14         Console.WriteLine("Hello from Singleton Pattern");
15     }
16 }
17 class Program
18 {
19     static void Main(string[] args){
20         Singleton s1 = Singleton.GetInstance();
21         Singleton s2 = Singleton.GetInstance();
22         s1.Display();
23         if (s1 == s2){
24             Console.WriteLine("Only One Instance Exists");
25         }
26         Console.ReadLine();
27     }
28 }
```

Week-1: Design and Patterns and Principles

2) Factory method Pattern:

- ❑ Factory Method Pattern provides an interface for creating objects without specifying their exact class.
- ❑ Object creation logic is centralized in a factory class, reducing code duplication.
- ❑ The client code depends on abstractions (interfaces) rather than concrete classes.
- ❑ New object types can be added with minimal changes, supporting the Open-Closed Principle (OCP).
- ❑ Factory Method improves flexibility, maintainability, and scalability.

```
1  using System;
2  public interface IDocument{
3      void Open();
4  }
5  public class WordDocument{
6      public void Open(){
7          Console.WriteLine("Opening Word Document");
8      }
9  }
10 public class PdfDocument{
11     public void Open(){
12         Console.WriteLine("Opening PDF Document");
13     }
14 }
15 public class DocumentFactory{
16     public static IDocument CreateDocument(string type){
17         if (type.Equals("Word", StringComparison.OrdinalIgnoreCase)){
18             return new WordDocument();
19         }
20         else if (type.Equals("PDF", StringComparison.OrdinalIgnoreCase)){
21             return new PdfDocument();
22         }
23         throw new ArgumentException("Invalid document type");
24     }
25 }
26 class Program{
27     static void Main(string[] args){
28         IDocument doc1 = DocumentFactory.CreateDocument("Word");
29         doc1.Open();
30         IDocument doc2 = DocumentFactory.CreateDocument("PDF");
31         doc2.Open();
32         Console.ReadLine();
33     }
34 }
```

Week-1: Design and Patterns and Principles

3) Builder Pattern:

- The Builder Pattern is a **creational design pattern** used to construct complex objects step by step.
- It helps avoid **telescoping constructors** by providing flexible object creation with optional parameters.
- It improves **readability and maintainability** by separating object construction from its representation.
- It uses a **Builder class (often static nested)** that returns the final object using a Build() method.

```
1  using System;
2  class Computer{
3      public string CPU, RAM, Storage;
4      private Computer(Builder b) { CPU = b.CPU; RAM = b.RAM; Storage = b.Storage; }
5      public class Builder{
6          public string CPU, RAM, Storage;
7          public Builder SetCPU(string v) { CPU = v; return this; }
8          public Builder SetRAM(string v) { RAM = v; return this; }
9          public Builder SetStorage(string v) { Storage = v; return this; }
10         public Computer Build() => new Computer(this);
11     }
12     public void Show() => Console.WriteLine($"{CPU}, {RAM}, {Storage}");
13 }
14 class Program{
15     static void Main(){
16         var pc = new Computer.Builder()
17             .SetCPU("i7")
18             .SetRAM("16GB")
19             .SetStorage("512GB SSD")
20             .Build();
21         pc.Show();
22     }
23 }
```

Week-1: Design and Patterns and Principles

4) Adapter Pattern:

- The Adapter Pattern is a **structural design pattern** used to make incompatible interfaces work together.
- It acts as a **bridge between a client and a third-party/legacy system** without modifying existing code.
- It converts one interface into another expected by the client (via an **Adapter class**).
- It is widely used in **payment systems, APIs, and integrations** where multiple external services differ.

```
1  using System;
2  interface IPaymentProcessor{
3      void ProcessPayment(double amount);
4  }
5  class PayPal{
6      public void SendPayment(double amt) =>
7          Console.WriteLine($"PayPal payment of {amt}");
8  }
9  class RazorPay{
10     public void MakeTransaction(double amt) =>
11         Console.WriteLine($"RazorPay payment of {amt}");
12 }
13 class PayPalAdapter{
14     PayPal pp = new PayPal();
15     public void ProcessPayment(double amt) => pp.SendPayment(amt);
16 }
17 class RazorAdapter{
18     RazorPay rp = new RazorPay();
19     public void ProcessPayment(double amt) => rp.MakeTransaction(amt);
20 }
21 class Program{
22     static void Main(){
23         IPaymentProcessor p1 = new PayPalAdapter();
24         IPaymentProcessor p2 = new RazorAdapter();
25         p1.ProcessPayment(500);
26         p2.ProcessPayment(1000);
27     }
28 }
```

Week-1: Design and Patterns and Principles

5) Decorator Pattern:

- The Decorator Pattern is a **structural design pattern** used to dynamically add behavior to objects without changing their structure.
- It follows **composition over inheritance**, allowing flexible feature extension at runtime.
- It wraps objects inside other objects (decorators) that add extra functionality.
- Commonly used in **notification systems, UI components, and I/O streams**.

```
1  using System;
2  interface INotifier { void Send(string m); }
3  class Email : INotifier{
4  |   public void Send(string m) => Console.WriteLine("Email->" + m + " ");
5  | }
6  class Decorator : INotifier{
7  |   protected INotifier n;
8  |   public Decorator(INotifier n) { this.n = n; }
9  |   public virtual void Send(string m) => n.Send(m);
10 | }
11 class SMS : Decorator{
12 |   public SMS(INotifier n) : base(n) { }
13 |   public override void Send(string m) { base.Send(m); Console.WriteLine("SMS->" + m + " "); }
14 | }
15 class Slack : Decorator{
16 |   public Slack(INotifier n) : base(n) { }
17 |   public override void Send(string m) { base.Send(m); Console.WriteLine("Slack->" + m + " "); }
18 | }
19 class Program{
20 |   static void Main(){
21 |       INotifier n = new Slack(new SMS(new Email()));
22 |       n.Send("Hi");
23 |   }
24 }
```

Week-1: Design and Patterns and Principles

6) Proxy Pattern:

- The Proxy Pattern is a **structural design pattern** that provides a surrogate (placeholder) for another object.
- It controls access to the real object and can add features like **lazy loading, caching, or security checks**.
- The real object is created only when needed (**lazy initialization**).
- Commonly used in **image loading, database access, and remote services**.

```
1  using System;
2  interface IImage { void Display(); }
3  class RealImage : IImage{
4      string file;
5      public RealImage(string f) { file = f; Load(); }
6      void Load() => Console.WriteLine("Loading " + file);
7      public void Display() => Console.WriteLine("Displaying " + file);
8  }
9  class ProxyImage : IImage{
10     RealImage img;
11     string file;
12     public ProxyImage(string f) { file = f; }
13     public void Display(){
14         if (img == null) img = new RealImage(file);
15         img.Display();
16     }
17 }
18 class Program{
19     static void Main()
20     {
21         IImage i = new ProxyImage("photo.jpg");
22         i.Display();
23         i.Display();
24     }
25 }
```

Week-1: Design and Patterns and Principles

7) Observer Pattern:

- The Observer Pattern is a **behavioral design pattern** used for one-to-many communication between objects.
- When the subject changes state, all registered observers are **automatically notified**.
- It follows a **publish-subscribe model** (loose coupling between subject and observers).
- Commonly used in **stock systems, weather apps, and event listeners**.

```
1  using System;
2  using System.Collections.Generic;
3  interface IObservable { void Update(float price); }
4  interface IStock{
5      void Register(IObservable o);
6      void Deregister(IObservable o);
7      void Notify();
8  }
9  class StockMarket : IStock{
10     List<IObservable> obs = new();
11     float price;
12     public void SetPrice(float p) { price = p; Notify(); }
13     public void Register(IObservable o) => obs.Add(o);
14     public void Deregister(IObservable o) => obs.Remove(o);
15     public void Notify(){
16         foreach (var o in obs) o.Update(price);
17     }
18 }
19 class MobileApp : IObservable{
20     public void Update(float p) => Console.WriteLine("Mobile: " + p);
21 }
22 class WebApp : IObservable{
23     public void Update(float p) => Console.WriteLine("Web: " + p);
24 }
25 class Program{
26     static void Main(){
27         StockMarket sm = new();
28         sm.Register(new MobileApp());
29         sm.Register(new WebApp());
30         sm.SetPrice(120.5f);
31         sm.SetPrice(130.0f);
32     }
33 }
```

Week-1: Design and Patterns and Principles

8) Strategy Pattern:

- The Strategy Pattern is a **behavioral design pattern** that allows selecting an algorithm at runtime.
- It defines a **family of algorithms** and makes them interchangeable.
- It follows **composition over inheritance** for flexible behavior switching.
- Commonly used in **payment systems, sorting algorithms, and navigation apps**.

```

1 using System;
2 interface IPaymentStrategy { void Pay(int amt); }
3 class CreditCard : IPaymentStrategy{
4     public void Pay(int amt) => Console.WriteLine("CreditCard: " + amt);
5 }
6 class PayPal : IPaymentStrategy{
7     public void Pay(int amt) => Console.WriteLine("PayPal: " + amt);
8 }
9 class PaymentContext{
10     IPaymentStrategy s;
11     public void SetStrategy(IPaymentStrategy s) { this.s = s; }
12     public void Pay(int amt) => s.Pay(amt);
13 }
14 class Program{
15     static void Main(){
16         PaymentContext p = new PaymentContext();
17         p.SetStrategy(new CreditCard());
18         p.Pay(500);
19         p.SetStrategy(new PayPal());
20         p.Pay(800);
21     }
22 }

```


Week-1: Design and Patterns and Principles

9) Command Pattern:

- The Command Pattern is a **behavioral design pattern** that encapsulates a request as an object.
- It separates **sender (Invoker)** and **receiver**, increasing flexibility and decoupling.
- It allows **undo/redo, logging, and queuing of requests**.
- Commonly used in **remote controls, task scheduling, and UI button actions**.

```
1  using System;
2  interface ICommand { void Execute(); }
3  class Light{
4      public void On() => Console.WriteLine("Light ON");
5      public void Off() => Console.WriteLine("Light OFF");
6  }
7  class LightOn : ICommand{
8      Light l;
9      public LightOn(Light l) { this.l = l; }
10     public void Execute() => l.On();
11 }
12 class LightOff : ICommand{
13     Light l;
14     public LightOff(Light l) { this.l = l; }
15     public void Execute() => l.Off();
16 }
17 class RemoteControl{
18     ICommand cmd;
19     public void Set(ICommand c) { cmd = c; }
20     public void Press() => cmd.Execute();
21 }
22 class Program{
23     static void Main(){
24         Light l = new Light();
25         RemoteControl r = new RemoteControl();
26         r.Set(new LightOn(l));
27         r.Press();
28         r.Set(new LightOff(l));
29         r.Press();
30     }
31 }
```

Week-1: Design and Patterns and Principles

10)MVC Pattern:

- MVC (**Model-View-Controller**) separates an application into three components: Model, View, and Controller.
- **Model** stores and manages data, **View** displays data, and **Controller** handles user interactions.
- It improves **maintainability, scalability, and code organization**.
- Commonly used in **web applications, desktop applications, and mobile apps**.

```
1  using System;
2  class Student{
3      public string Name { get; set; }
4      public int Id { get; set; }
5      public string Grade { get; set; }
6  }
7  class StudentView{
8      public void Display(Student s) =>
9          Console.WriteLine($"Name:{s.Name} Id:{s.Id} Grade:{s.Grade}");
10 }
11 class StudentController{
12     Student m; StudentView v;
13     public StudentController(Student m, StudentView v)
14     { this.m = m; this.v = v; }
15     public void SetName(string n) => m.Name = n;
16     public void UpdateView() => v.Display(m);
17 }
18 class Program{
19     static void Main(){
20         Student s = new Student { Name = "John", Id = 101, Grade = "A" };
21         StudentController c = new StudentController(s, new StudentView());
22         c.SetName("David");
23         c.UpdateView();
24     }
25 }
```

Week-1: Design and Patterns and Principles

11) Dependency Injection (DI):

- Dependency Injection is a design principle where dependencies are provided from outside rather than created inside a class.
- It promotes **loose coupling** between components, making code easier to maintain and test.
- **Constructor Injection** is the most common form of DI.
- Widely used in frameworks like Spring Framework and ASP.NET Core.

```
1  using System;
2  interface ICustomerRepository{
3      string FindCustomerById(int id);
4  }
5  class CustomerRepository : ICustomerRepository{
6      public string FindCustomerById(int id) => "Customer ID: " + id;
7  }
8  class CustomerService{
9      ICustomerRepository repo;
10     public CustomerService(ICustomerRepository repo){
11         this.repo = repo;
12     }
13     public void GetCustomer(int id){
14         Console.WriteLine(repo.FindCustomerById(id));
15     }
16 }
17 class Program{
18     static void Main(){
19         ICustomerRepository repo = new CustomerRepository();
20         CustomerService service = new CustomerService(repo);
21         service.GetCustomer(101);
22     }
23 }
```